

applications. The traditional spider explores the response HTML, while the Ajax spider invokes browsers and then follows the links that are given out. We can use either one of these or can use both in our penetration testing, as shown in Figure 8.

10. In the *Finalize run* section, select *Generate Reports* to get the results of the attack previously conducted, and select the type of reports you want to get. It has two options — XML or HTML type of reports or both can be published, as seen in Figure 9. Save the configurations.
11. In the post-build step, select *Archive the artifacts*, publish the *HTML reports* option and fill in the details, as shown in Figure 10. Here, we can name the HTML reports as per our needs as well as the folder where it is being published.

The result

After building the project, we get the options to open the reports of our choice in the *Stage view* area. Reports can be accessed by selecting the links below the last successful artefact's icon, which is shown in Figure 11.

From the generated report, we can find a detailed summary of the type of attacks done, the number of vulnerabilities found at various degrees of threat levels, and an explanation of the vulnerability along with the solutions for it. Figure 12 shows the ZAP scanning report for our Web application.

From the report, we can infer that the threat levels of the vulnerabilities are divided into four categories, namely, High, Medium, Low and Informational which are provided to us in the form of a table. For example, in the above case we found one medium level threat and four low level threats in our Web application.

The report also gives a detailed account of each type of vulnerability found. It provides a description of the vulnerability parameters as well as the

Figure 8: Attack mode

Figure 9: Finalize run

Figure 10: Post-build actions